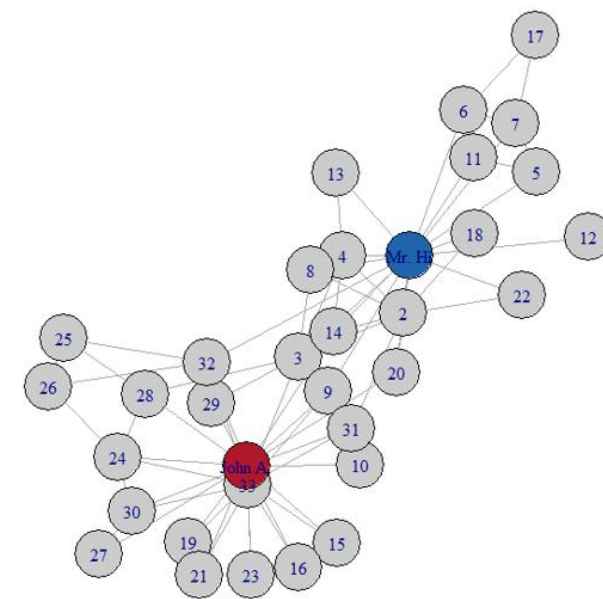
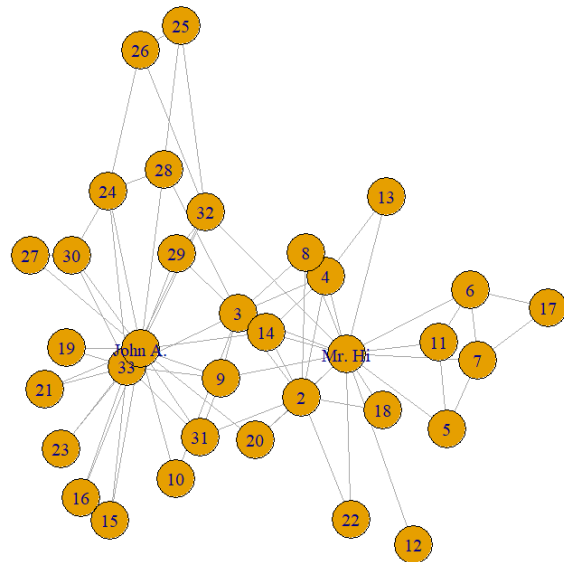


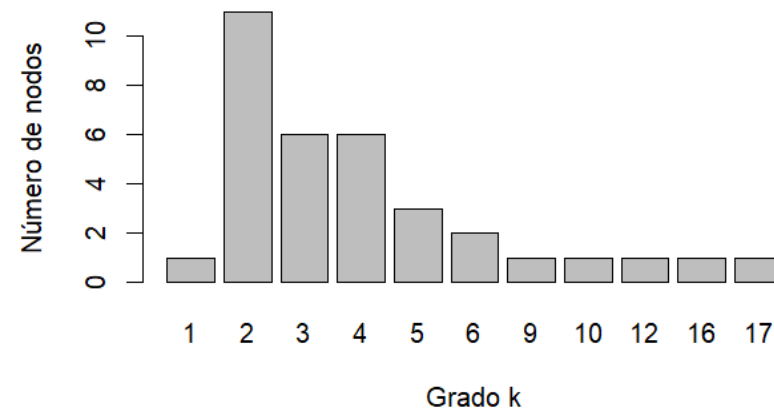
Centralidad, cohesión, comunidades, roles y embeddings

Centralidad · Cohesión · Comunidades · Roles estructurales · Graph embeddings

De la posición individual del nodo a su representación como vector en un espacio estructural.



Distribución de grado - Zachary Karate Club



¿Qué significa ocupar una posición importante en una red?

“Importancia” no es una propiedad única

Un nodo puede ser importante porque:

- Tiene muchas conexiones;
- Conecta regiones separadas;
- Está cerca del resto de la red;
- Se conecta con actores importantes;
- Pertenece al núcleo de una estructura;
- Ocupa una posición estructural poco frecuente.

Por tanto:

centralidad $\neq$ una única propiedad

▮ Cada métrica formaliza una idea distinta de importancia.

La posición depende del proceso que nos interesa

Supongamos que queremos identificar:

- Quién puede difundir información rápidamente;
- Quién controla pasos entre grupos;
- Quién tiene mayor visibilidad;
- Quién posee mayor prestigio;
- Quién está estructuralmente integrado.

Cada pregunta requiere una definición diferente de centralidad.

Borgatti (2005):

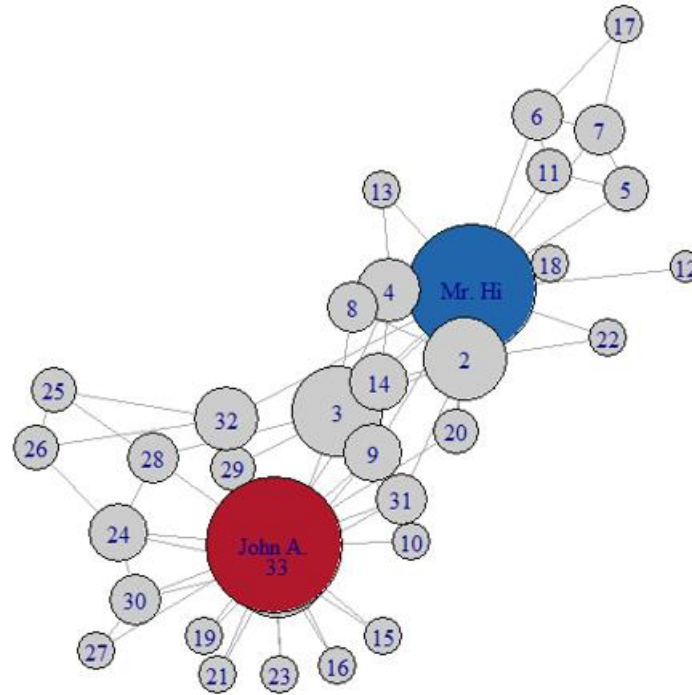
la interpretación correcta de una medida de centralidad depende del proceso que suponemos circulando por la red.

Centralidad de grado

La medida más inmediata de centralidad es el número de conexiones.

Para una red no dirigida:

$$C_D(i) = k_i = \sum_j A_{ij}$$



		nodo	grado
John A.	John A.	17	
Mr. Hi	Mr. Hi	16	
33	33	12	
3	3	10	
2	2	9	
4	4	6	
32	32	6	
9	9	5	
14	14	5	
24	24	5	

Un nodo con alto grado posee muchos contactos directos.

Grado normalizado

Para comparar redes de distintos tamaños:

$$C_D^{norm}(i) = \frac{k_i}{n-1}$$

Entonces:

$$0 \leq C_D^{norm}(i) \leq 1$$

Un valor cercano a 1 indica que el nodo está conectado directamente con gran parte de la red.

Redes dirigidas

En redes dirigidas distinguimos:

In-degree

$$C_D^{in}(i) = \sum_j A_{ji}$$

Puede representar:

- Popularidad;
- Prestigio;
- Recepción de información.

Out-degree

$$C_D^{out}(i) = \sum_j A_{ij}$$

Puede representar:

- Actividad;
- Búsqueda;
- Emisión de información.

CENTRALIDAD DE GRADO

¿Qué representa el grado?

El grado puede interpretarse como:

Acceso inmediato

¿Cuántos actores puedo
alcanzar directamente?

Visibilidad

¿Cuántos actores están
vinculados conmigo?

Actividad

¿Cuántas relaciones
mantengo?

Pero sólo considera **vínculos directos**.

No incorpora la posición del nodo dentro de la estructura completa.

Intermediación

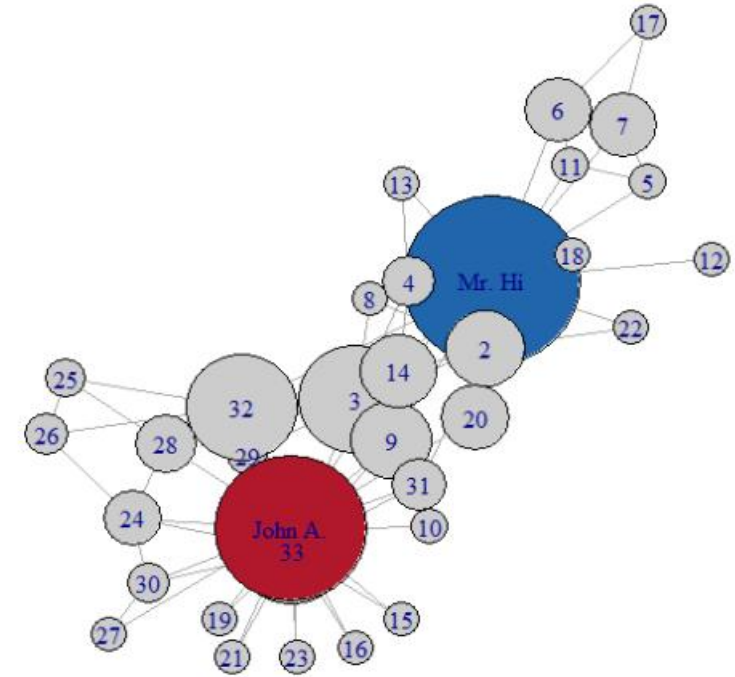
Un actor puede tener pocos vínculos y ser crucial porque conecta regiones de la red.

La centralidad de intermediación mide cuántos caminos geodésicos pasan por un nodo.

$$C_B(i) = \sum_{s \neq i \neq t} \frac{\sigma_{st}(i)}{\sigma_{st}}$$

donde:

- σ_{st} : número de caminos geodésicos entre s y t ;
- $\sigma_{st}(i)$: número de esos caminos que pasan por i .



	nodo	grado	cercania	intermediacion
Mr. Hi	Mr. Hi	16	0.5689655	231.07143
John A.	John A.	17	0.5500000	160.55159
33	33	12	0.5156250	76.69048
3	3	10	0.5593220	75.85079
32	32	6	0.5409836	73.00952
9	9	5	0.5156250	29.52937
2	2	9	0.4852941	28.47857
14	14	5	0.5156250	24.21587
20	20	3	0.5000000	17.14683
6	6	4	0.3837209	15.83333

Intuición de la intermediación

Considere:



El nodo X puede tener **grado bajo**. Sin embargo, muchos caminos entre ambos grupos pasan por él.

Por tanto: $C_D(X)$ puede ser **moderado**, mientras $C_B(X)$ puede ser **muy alto**.

Intermediación y brokerage

Una alta intermediación puede representar:

- Capacidad de *brokerage*;
- Control de flujos;
- Acceso a información no redundante;
- Vulnerabilidad sistémica;
- Capacidad de conectar comunidades.

Pero la interpretación supone que el proceso relevante utiliza caminos relativamente cortos.

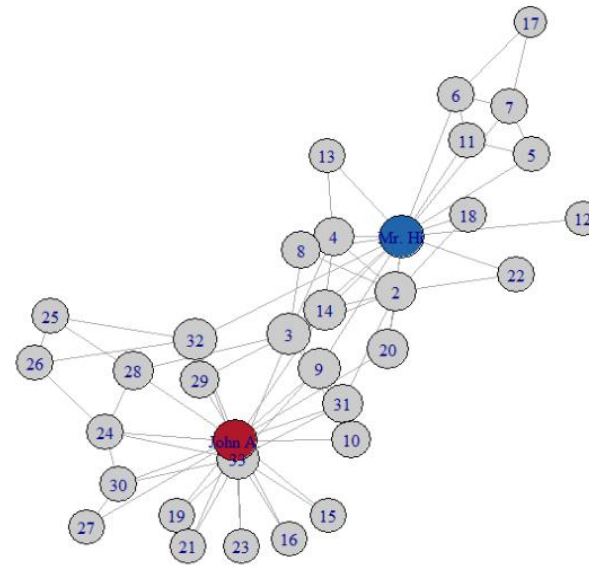
Cercanía

La centralidad de cercanía mide cuán cerca está un nodo del resto.

Una formulación clásica es:

$$C_C(i) = \frac{n-1}{\sum_{j \neq i} d(i,j)}$$

donde $d(i,j)$ es la distancia geodésica entre i y j .



```
> cercania_manual
[1] 0.5689655 0.4852941 0.5593220 0.4647887 0.3793103 0.3837209 0.3837209 0.4400000 0.5156250 0.4342105 0.3793103
[12] 0.3666667 0.3707865 0.5156250 0.3707865 0.3707865 0.2844828 0.3750000 0.3707865 0.5000000 0.3707865 0.3750000
[23] 0.3707865 0.3928571 0.3750000 0.3750000 0.3626374 0.4583333 0.4520548 0.3837209 0.4583333 0.5409836 0.5156250
[34] 0.5500000
```

Intuición de la cercanía

Un nodo tiene alta cercanía cuando necesita pocos pasos para alcanzar al resto.

Puede interpretarse como:

- Rapidez potencial de acceso;
- Eficiencia de comunicación;
- Capacidad para alcanzar el sistema.

La lógica es:

$$\sum_j d(i, j) \downarrow \Rightarrow C_c(i) \uparrow$$

Un problema: redes desconectadas

Si i y j pertenecen a componentes distintos:

$$d(i, j) = \infty$$

La cercanía clásica presenta problemas.

Una alternativa es la **cercanía armónica**:

$$C_H(i) = \sum_{j \neq i} \frac{1}{d(i, j)}$$

definiendo $\frac{1}{\infty} = 0$

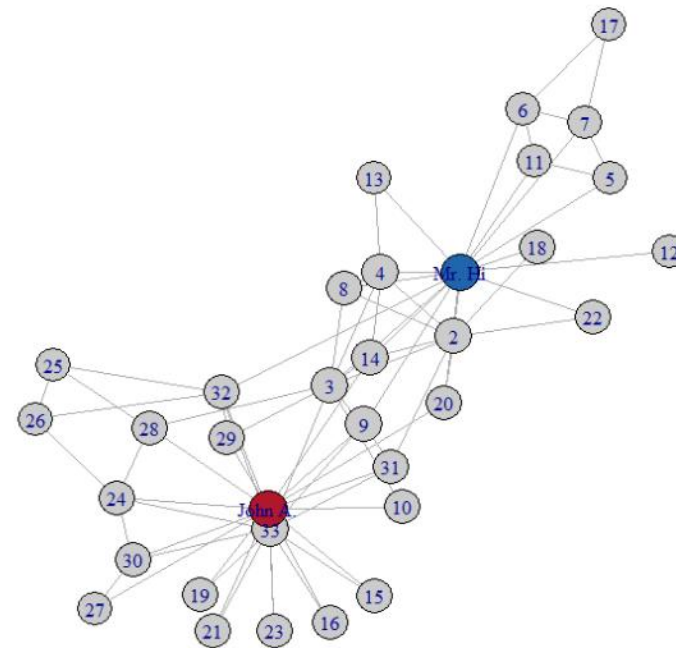
No todas las conexiones valen lo mismo

La centralidad de grado cuenta conexiones.

Eigenvector centrality introduce otra idea: estar conectado con actores importantes aumenta la propia importancia.

Formalmente:

$$x_i = \frac{1}{\lambda} \sum_j A_{ij} x_j$$



Forma matricial

La expresión anterior puede escribirse como:

$$A x = \lambda x$$

Por tanto, **x** es un **eigenvector** de la matriz de adyacencia.

La centralidad de eigenvector utiliza habitualmente el eigenvector asociado al mayor eigenvalue: λ_{max}

Prestigio recursivo

Dos actores pueden tener el mismo grado:

$$k_A = k_B$$

Pero si A está conectado con actores muy centrales y B con actores periféricos:

$$C_E(A) > C_E(B)$$

La importancia depende de quiénes son nuestros vecinos.

Cuatro ideas distintas de centralidad

	grado	intermediacion	cercania	eigenvector
grado	1.0000000	0.9146430	0.7715910	0.9172546
intermediacion	0.9146430	1.0000000	0.7179446	0.8032146
cercania	0.7715910	0.7179446	1.0000000	0.9046179
eigenvector	0.9172546	0.8032146	0.9046179	1.0000000

Medida	Pregunta
Grado	¿Cuántos contactos directos tengo?
Intermediación	¿Cuántos caminos pasan por mí?
Cercanía	¿Qué tan cerca estoy de todos?
Eigenvector	¿Estoy conectado con actores importantes?

No existe “el nodo más central” sin especificar primero qué significa centralidad.

Ejercicio breve

Considere una red formada por:

- Dos grupos densamente conectados;
- Un nodo que conecta ambos grupos;
- Un nodo muy popular dentro del primer grupo;
- Un nodo conectado sólo con el nodo más popular.

Prediga cuál tendrá mayor:

- Grado;
- Intermediación;
- Cercanía;
- Eigenvector centrality.

COHESIÓN

Del nodo al grupo

La centralidad estudia principalmente posiciones individuales.

La **cohesión** pregunta: ¿qué tan fuertemente conectado está un conjunto de nodos?

Podemos estudiar cohesión mediante:

- Densidad;
- Caminos;
- Cliques;
- k-cores;
- Conectividad.

COHESIÓN

Cliques

Un **clique** es un subconjunto de nodos donde todos están conectados con todos.

Para un conjunto S :

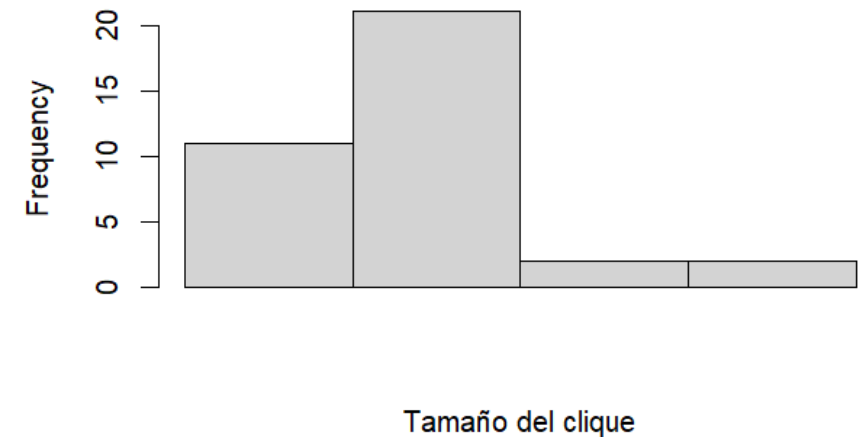
$$A_{ij} = 1 \quad \forall i, j \in S, i \neq j$$

Un clique de tamaño k contiene $\frac{k(k-1)}{2}$ aristas.

n-clique vs n-clan

```
> cliques_maximos ← max_cliques(g)
> length(cliques_maximos)
[1] 36
```

Distribución del tamaño de los cliques



COHESIÓN

Por qué interesan los cliques

Un clique representa una forma extrema de cohesión local.

Puede corresponder a:

- Grupos fuertemente cerrados;
- Equipos;
- Círculos de confianza;
- Coaliciones altamente integradas.

Pero exigir conexión completa puede ser demasiado restrictivo para redes reales.

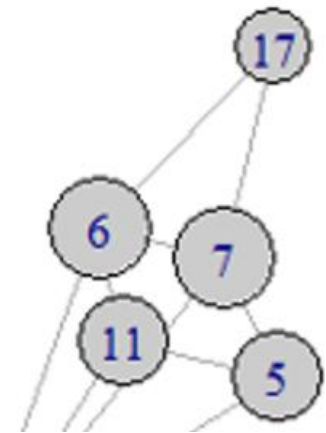
COHESIÓN

k-cores

Un **k-core** es un subgrafo maximal en el que cada nodo tiene al menos k conexiones con otros nodos del mismo subgrafo.

Formalmente:

$$k_i^{(S)} \geq k \text{ para todo } i \in S$$



Descomposición por cores

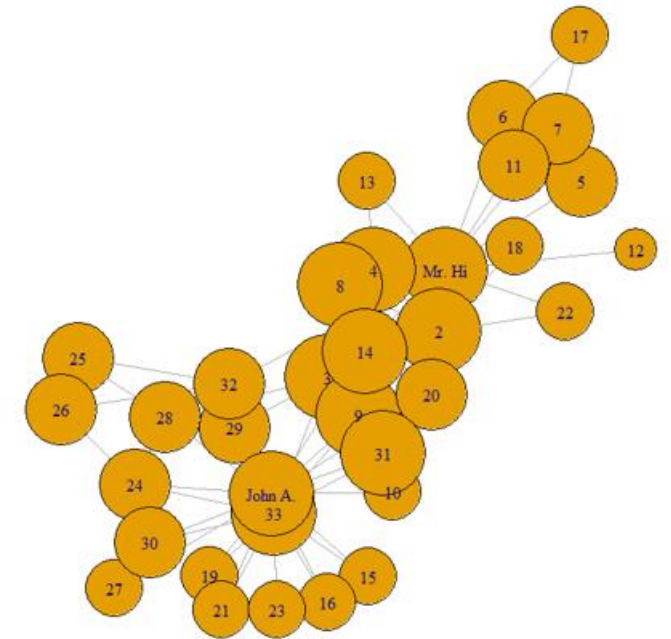
Podemos eliminar iterativamente:

- Nodos con grado menor que k ;
- Recalcular grados;
- Repetir hasta que ningún nodo incumpla la condición.

Esto permite distinguir:

- Periferia;
- Regiones intermedias;
- Núcleo estructural.

El máximo k al que pertenece un nodo se denomina su **coreness**.



Grado y coreness son diferentes

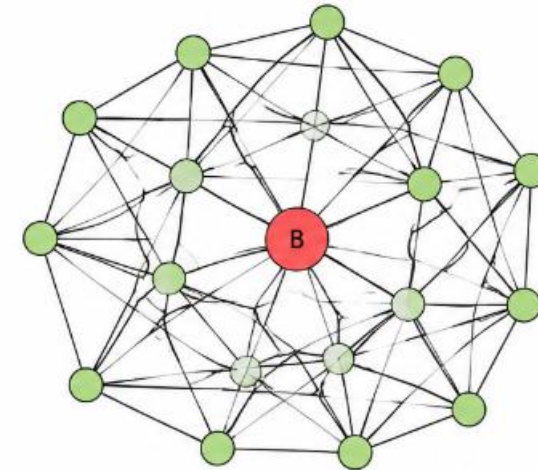
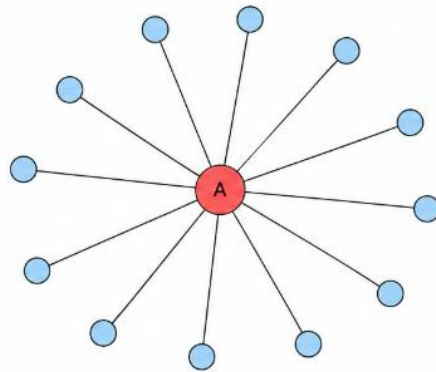
Un nodo puede tener **alto grado** porque conecta con muchos nodos periféricos.

Otro puede tener menos vínculos, pero pertenecer a un núcleo altamente interconectado.

Por tanto:

grado alto $\nRightarrow$ core alto

+ Popular



+ Pertenencia

La posición depende también de cómo están conectados sus vecinos.

¿Qué es una comunidad?

Intuitivamente, una comunidad es un conjunto de nodos:

- Fuertemente conectados entre sí;
- Relativamente menos conectados con el resto.

Una partición puede representarse como:

$$C = \{C_1, C_2, \dots, C_K\}$$

Comunidades como estructura meso

Las comunidades ocupan un nivel intermedio entre:

nodos → comunidades → red completa

Pueden corresponder a:

- Grupos sociales;
- Partidos;
- Disciplinas;
- Mercados;
- Regiones;
- Comunidades puramente estructurales.

La interpretación sustantiva debe establecerse después de identificarlas.

Modularidad

La **modularidad** compara la cantidad de conexiones dentro de comunidades con la esperada bajo un modelo nulo.

Una formulación clásica es:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

Newman, M. E. J. (2006). "Modularity and community structure in networks." Proceedings of the National Academy of Sciences, 103(23), 8577-8582.

¿Qué compara Q?

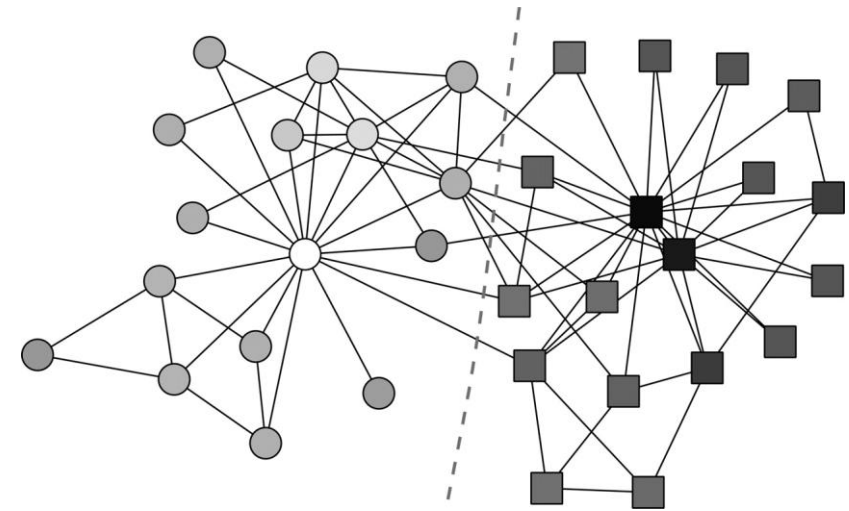
El término A_{ij}
representa la conexión **observada**.

El término $\frac{k_i k_j}{2m}$
representa la conexión **esperada** bajo un modelo
nulo que preserva grados.

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

Y: $\delta(c_i, c_j) = 1$

cuando i y j pertenecen a la misma
comunidad.



Newman, M. E. J. (2006)

Interpretación de modularidad

Una partición tiene **modularidad alta** cuando existen más vínculos dentro de comunidades que los esperados bajo el modelo nulo.

$Q \uparrow$ implica **mayor separación estructural** entre comunidades respecto del contrafactual.

Pero Q no mide simplemente “qué buena es la sociedad” ni “qué cohesionado está cada grupo”.

El problema computacional

Dada una red $G = (V, E)$ queremos encontrar una partición C^* que revele una estructura comunitaria relevante.

El número de particiones posibles crece **extremadamente rápido** con n .

| Por eso utilizamos algoritmos aproximados.

Múltiples alternativas, algunas de ellas:

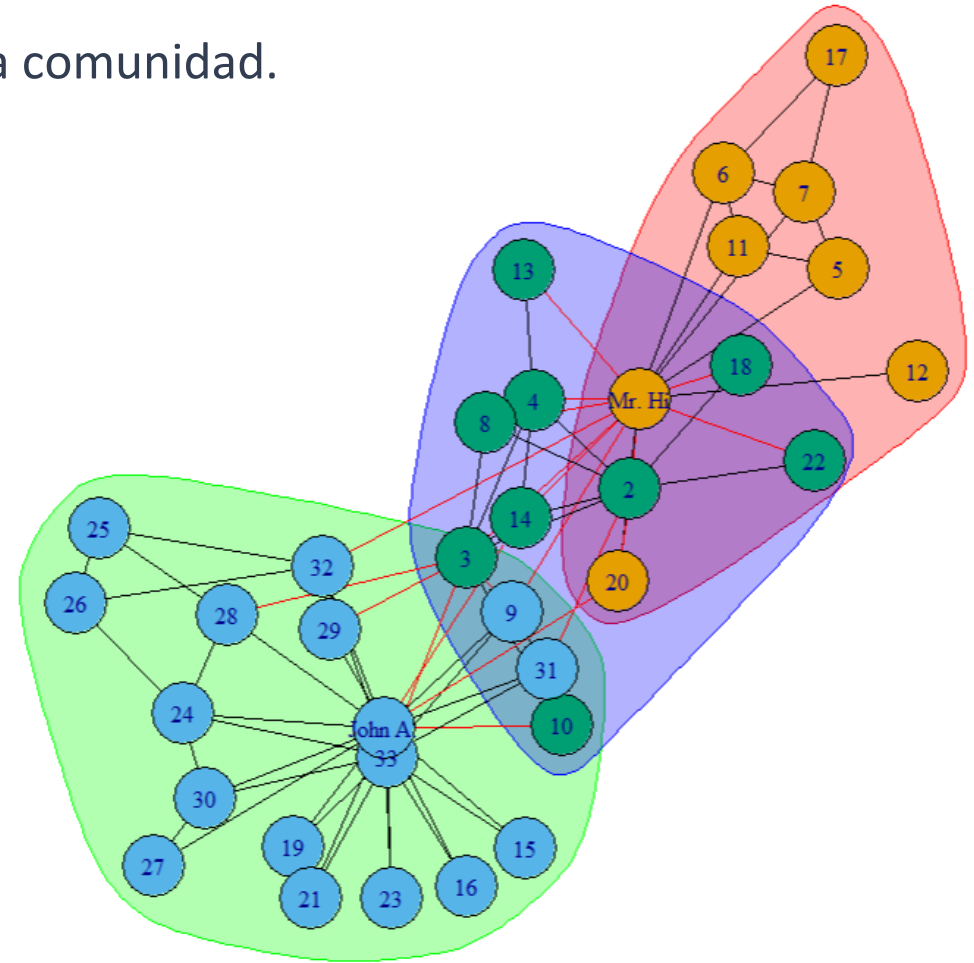
Método	La idea
Girvan-Newman / Edge betweenness	Elimina sucesivamente las aristas que más conectan regiones distintas de la red.
Fast Greedy	Fusiona grupos paso a paso buscando aumentar la modularidad en cada iteración.
Louvain	Mueve nodos entre comunidades y luego agrega comunidades completas para maximizar modularidad jerárquicamente.
Leiden	Similar a Louvain, pero refina las comunidades para evitar grupos internamente mal conectados.
Walktrap	Los random walks cortos tienden a permanecer dentro de una comunidad antes de salir de ella.
Infomap	Busca grupos donde un caminante aleatorio permanece mucho tiempo, comprimiendo la descripción de su trayectoria.
Label Propagation	Cada nodo adopta la etiqueta más frecuente entre sus vecinos hasta que emergen grupos relativamente estables.
Spectral clustering	Usa los autovectores de una matriz de la red para identificar separaciones estructurales entre grupos.
Stochastic Block Model (SBM)	Supone que los nodos pertenecen a bloques que tienen distintas probabilidades de conectarse entre sí.
Degree-Corrected SBM	Extiende el SBM permitiendo que los nodos tengan grados muy distintos dentro de un mismo bloque.
Clique Percolation	Construye comunidades a partir de cliques adyacentes que comparten nodos. Permite comunidades solapadas.
OSLOM	Busca grupos cuya densidad interna sería improbable bajo un modelo nulo que considera los grados.
Leading Eigenvector	Divide la red usando el principal autovector de la matriz de modularidad.
Spinglass	Trata la detección de comunidades como un problema de minimización de energía inspirado en modelos de física estadística.

Greedy modularity

Greedy modularity comienza con cada nodo en su propia comunidad.

Luego fusiona iterativamente las comunidades cuya unión produce el mayor aumento en Q .

El proceso continúa hasta que nuevas fusiones dejan de mejorar la modularidad.



Louvain

Louvain busca maximizar modularidad mediante dos pasos repetidos.

Paso 1

Mover nodos entre comunidades buscando

$$\Delta Q > 0$$

Luego repetir.

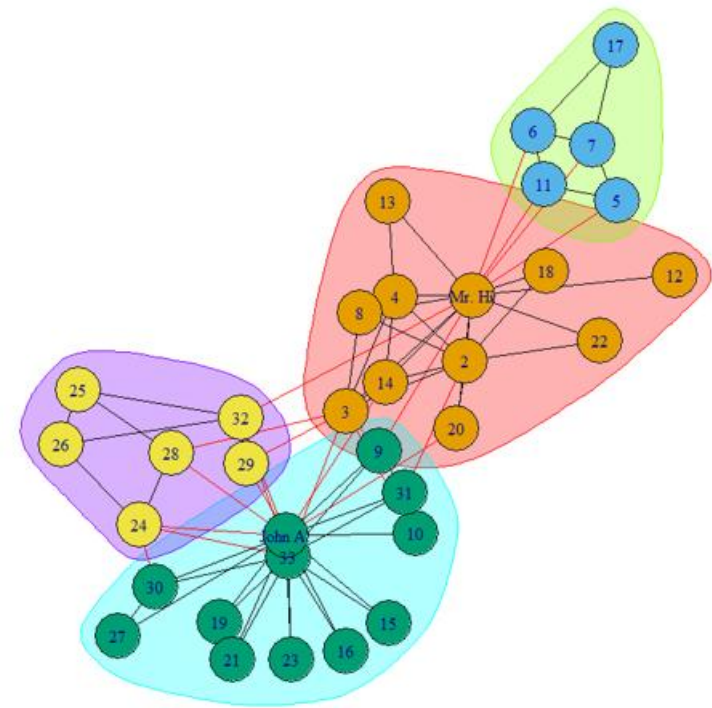
Paso 2

Agregar cada comunidad detectada como un nuevo **supernodo**.

Intuición de Louvain

nodos $\rightarrow$ comunidades locales $\rightarrow$ supernodos $\rightarrow$ nuevas comunidades

La estructura se construye **jerárquicamente**.



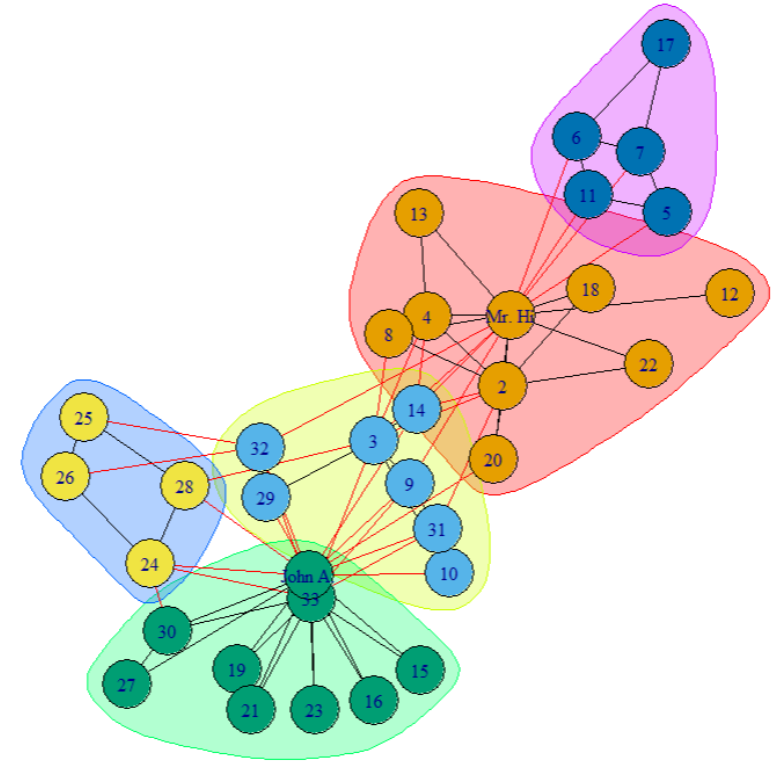
Louvain es eficiente y ampliamente utilizado para redes grandes.

Walktrap

Walktrap parte de una intuición diferente:

Una caminata aleatoria tiende a permanecer dentro de una comunidad antes de escapar hacia otra.

Si dos nodos generan distribuciones similares después de una caminata corta, probablemente pertenecen a regiones estructuralmente similares.



Los algoritmos pueden producir particiones diferentes

Louvain, greedy modularity y walktrap:

- Utilizan criterios diferentes;
- Pueden producir distinto número de comunidades;
- Pueden asignar algunos nodos de manera diferente.

La comunidad detectada no es una propiedad completamente independiente del algoritmo.

$$\textit{partición observada} = f (G , \textit{algoritmo} , \textit{parámetros})$$

	algoritmo	n_comunidades	modularidad
1	Louvain	4	0.4197896
2	Walktrap	5	0.3532216
3	Greedy modularity	3	0.3806706

Modularidad y resolución

La modularidad puede no detectar comunidades pequeñas cuando están insertas en redes grandes.

Este problema se conoce como **resolution limit**.

Una partición con mayor Q no necesariamente coincide con la escala sustantivamente relevante.

Detectar comunidades requiere combinar criterio estructural e interpretación sustantiva.

Comunidad y rol no son lo mismo

Dos nodos pueden pertenecer a comunidades diferentes y ocupar posiciones estructuralmente equivalentes.

Ejemplo:

- presidente de una organización;
- presidente de otra organización.

No están conectados a las mismas personas.

Pero pueden relacionarse de manera **similar** con sus respectivos entornos.

Equivalencia estructural

Dos nodos i y j son **estructuralmente equivalentes** si tienen relaciones similares con los mismos nodos.

Idealmente:

$$A_{ik} = A_{jk} \text{ para todos los demas nodos } k$$

En redes reales interesa habitualmente una equivalencia aproximada.

Rol estructural

Un **rol** describe el patrón de relaciones que caracteriza una posición.

Ejemplos:

- Broker;
- Hub;
- Periférico;
- Puente;
- Miembro del núcleo;
- Autoridad local.

La pregunta cambia de:

¿a qué grupo pertenece?

a:

¿qué función estructural desempeña?

Tres dimensiones distintas

Podemos distinguir:

Centralidad

¿Cuán prominente es el nodo?

Comunidad

¿Con qué región de la red está asociado?

Rol

¿Qué patrón estructural caracteriza su posición?

Un mismo nodo puede ser:

- Poco central globalmente;
- Central dentro de su comunidad;
- Estructuralmente importante como broker.

La comunidad nos dice dónde está el nodo; la centralidad, cuán prominente es; el rol, cómo está insertado dentro y entre comunidades.

¿Para qué sirven los embeddings?

Una representación vectorial facilita tareas como:

- Clasificación de nodos;
- Predicción de vínculos;
- Clustering;
- Detección de similitud;
- Visualización;
- Incorporación de redes en modelos estadísticos o de machine learning.

El embedding transforma estructura relacional en variables numéricas.

Un nodo ocupa muchas posiciones a la vez

Hasta ahora describimos un nodo mediante múltiples propiedades:

- grado;
- intermediación;
- cercanía;
- eigenvector centrality;
- coreness;
- comunidad;
- grado interno;
- participación entre comunidades.

Cada medida captura una dimensión distinta de su posición

Problema: a medida que acumulamos dimensiones, comparar nodos se vuelve más difícil.

La estructura de un nodo es multidimensional

Podemos imaginar cada nodo como una fila de una tabla:

Nodo	Grado	Betweenness	Coreness	(z) módulo	Participación	...
A	12	0.34	3	1.8	0.10	...
B	6	0.41	4	0.2	0.76	...
C	9	0.08	4	1.3	0.25	...

Pero ninguna de estas variables, por separado, resume toda la estructura relacional. Además, la propia matriz de adyacencia ya representa cada nodo mediante un vector de longitud n

Es decir, en una red con n nodos, cada nodo puede ser visto inicialmente como un punto en un espacio de dimensión n .

El problema: n dimensiones son demasiadas

Si tenemos una red con $n=10,000$ cada nodo podría ser representado por un vector de 10.000 posiciones.

Eso contiene mucha información, pero es:

- difícil de visualizar;
- costoso de modelar;
- redundante;
- poco práctico para comparar nodos.

Queremos una representación mucho más pequeña que conserve lo estructuralmente importante.

La idea de un graph embedding

Graph embeddings buscan representar cada nodo mediante un vector: $v_i \in \mathbb{R}^d$

Es decir transformar un objeto matemático desde $\mathbb{R}^n \rightarrow \mathbb{R}^d$, donde normalmente $d \ll n$

Por ejemplo: 10,000 dimensiones $\rightarrow$ 64 dimensiones

La tarea consiste en hacer esta compresión **sin destruir la estructura relevante de la red**.

similitud estructural $\rightarrow$ proximidad en el espacio vectorial



¿Qué propiedades de la estructura queremos conservar?

Dos nodos pueden ser similares porque:

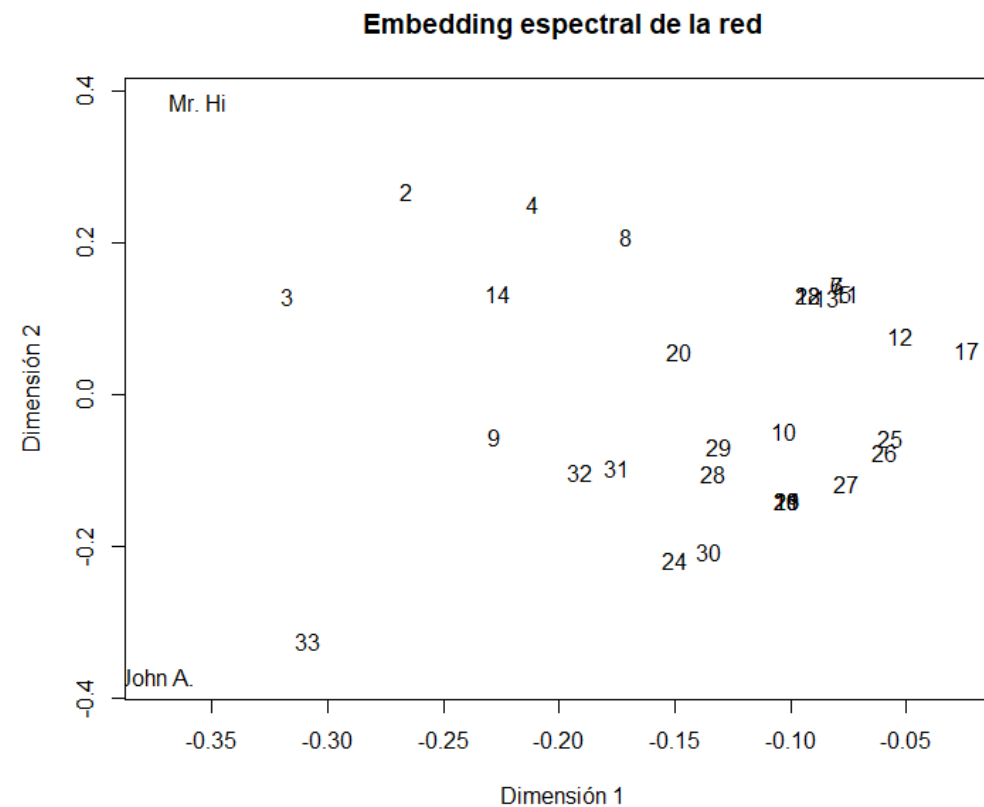
- están directamente conectados;
- tienen vecinos parecidos;
- pertenecen a la misma comunidad;
- ocupan roles estructurales semejantes;
- aparecen en contextos relacionales similares.

Por tanto, diferentes métodos de embedding pueden preservar **distintas nociones de similitud estructural**.

Algunos ejemplos

Método	Rasgo principal
DeepWalk	Usa random walks y trata las secuencias de nodos como frases para aprender proximidad estructural.
node2vec	Extiende DeepWalk controlando si los recorridos exploran vecindarios locales o regiones más amplias.
Laplacian Eigenmaps	Usa la estructura espectral del grafo para mantener próximos a nodos estructuralmente relacionados.
LINE	Preserva proximidad de primer y segundo orden y escala bien a redes grandes.
Graph Autoencoders	Aprenden representaciones comprimidas intentando reconstruir la estructura de la red.
GraphSAGE	Aprende el embedding de un nodo agregando información de sus vecinos, lo que permite generar representaciones para nodos nuevos.

Usando los dos eigenvectors principales de A sobre Zachary

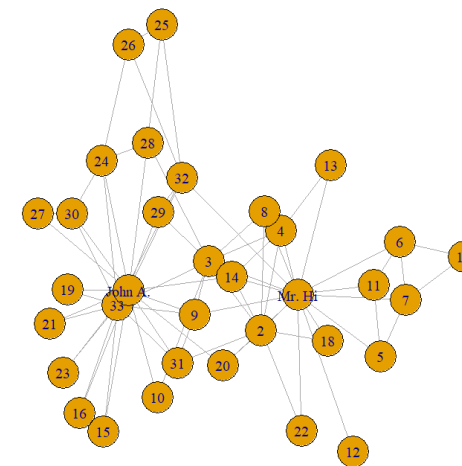


Intuición de Node2vec

Node2vec aprende representaciones de nodos a partir de caminatas aleatorias.

La lógica es:

- Generar secuencias de nodos mediante random walks;
- Tratar esas secuencias como “contextos”;
- Aprender qué nodos aparecen en contextos estructurales similares.



node2vec: Scalable Feature Learning for Networks

Aditya Grover, Jure Leskovec

Prediction tasks over nodes and edges in networks require careful effort in engineering features used by learning algorithms. Recent research in the broader field of representation learning has led to significant progress in automating prediction by learning the features themselves. However, present feature learning approaches are not expressive enough to capture the diversity of connectivity patterns observed in networks. Here we propose node2vec, an algorithmic framework for learning continuous feature representations for nodes in networks. In node2vec, we learn a mapping of nodes to a low-dimensional space of features that maximizes the likelihood of preserving network neighborhoods of nodes. We define a flexible notion of a node's network neighborhood and design a biased random walk procedure, which efficiently explores diverse neighborhoods. Our algorithm generalizes prior work which is based on rigid notions of network neighborhoods, and we argue that the added flexibility in exploring neighborhoods is the key to learning richer representations. We demonstrate the efficacy of node2vec over existing state-of-the-art techniques on multi-label classification and link prediction in several real-world networks from diverse domains. Taken together, our work represents a new way for efficiently learning state-of-the-art task-independent representations in complex networks.

La analogía con lenguaje

En modelos de lenguaje:

palabras que aparecen en contextos similares
reciben representaciones similares.

En Node2vec:

nodos que aparecen en contextos de red
similares reciben representaciones similares.

secuencia de palabras $\leftrightarrow$ random walk en la red

Objetivo de Node2vec

Para cada nodo u , queremos maximizar la probabilidad de observar sus nodos de contexto:

$$\max_f \sum_{u \in V} \log P(N_S(u) \mid f(u))$$

donde:

- $f(u)$ es el embedding;
- $N_S(u)$ es el vecindario generado mediante una estrategia de caminata S .

Dos tipos de exploración

Node2vec controla las caminatas mediante dos parámetros: p y q . Estos permiten moverse entre dos lógicas.

Exploración local

Busca nodos dentro de una misma región.

Exploración estructural

Busca nodos que pueden ocupar roles similares aunque estén lejos entre sí.

BFS y DFS como intuición

Cuando las caminatas permanecen cerca del nodo de origen, se aproximan a una lógica tipo:

Breadth-First Search

Favorece similitud por **comunidad**.

Cuando exploran regiones más lejanas:

Depth-First Search

pueden favorecer similitud por **rol estructural**.

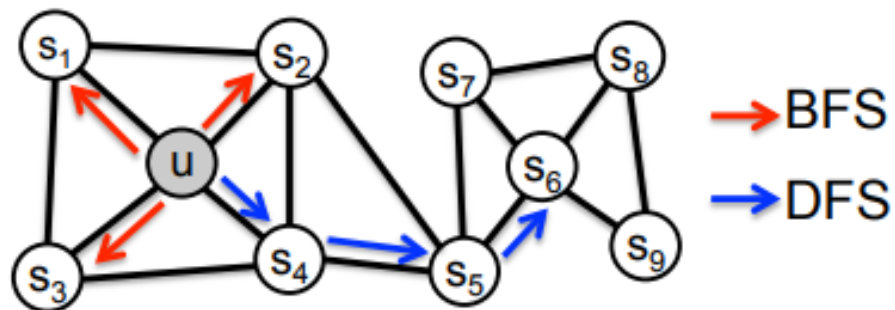


Figure 1: BFS and DFS search strategies from node u ($k = 3$).

Node2vec permite interpolar entre ambas.

Parámetro p

p controla la probabilidad de regresar al nodo anterior.

Un valor pequeño de p :

$$p < 1$$

favorece volver.

Un valor grande:

$$p > 1$$

desincentiva regresar inmediatamente.

Parámetro q

q controla cuánto se aleja la caminata del entorno inmediato.

De forma intuitiva:

$$q > 1$$

favorece exploración local.

$$q < 1$$

favorece exploración hacia regiones más lejanas.

Así podemos enfatizar comunidad o rol estructural.

Embedding no significa visualización

Node2vec puede producir:

$$v_i \in \mathbb{R}^{128}$$

Aunque luego proyectemos esos vectores a dos dimensiones para visualizarlos: $\mathbb{R}^{128} \rightarrow \mathbb{R}^2$

La representación original contiene más información que el gráfico bidimensional.

Qué puede revelar un embedding

Un embedding puede mostrar que:

- Miembros de una comunidad aparecen próximos;
- Brokers ocupan regiones intermedias;
- Hubs comparten patrones;
- Nodos distantes pueden tener roles similares.

Pero la interpretación depende del mecanismo utilizado para producir el embedding.

Una misma red, preguntas diferentes

Ante un nodo i :

Centralidad

$C(i)$

¿Cuán importante es?

Comunidad

c_i

¿A qué región
pertenece?

Cohesión

$core(i)$

¿Qué tan integrado
está?

Embedding

v_i

¿Qué nodos tienen
posiciones o contextos
similares?

Posición global y posición local

Un nodo puede tener:

- Grado moderado;
- Alta intermediación;
- Baja eigenvector centrality;
- Alto coreness;
- Pertenencia clara a una comunidad.

Estas medidas **no se contradicen**.

Describen dimensiones diferentes de la posición estructural.

Ejercicio de interpretación

Considere cuatro nodos:

Nodo A

Muchos vínculos dentro de una comunidad periférica.

Nodo C

Conectado con los nodos más prestigiosos.

Nodo B

Pocos vínculos, pero conecta dos comunidades.

Nodo D

Integrado en el núcleo más cohesivo.

Prediga qué nodo tendrá mayor: grado; intermediación; eigenvector centrality; coreness.

Referencias

Freeman, L. C. (1979). "Centrality in Social Networks: Conceptual Clarification." *Social Networks*, 1(3), 215-239.

<https://www.sciencedirect.com/science/article/pii/0378873378900217>

Borgatti, S. P. (2005). "Centrality and Network Flow." *Social Networks*, 27(1), 55-71.

<https://www.sciencedirect.com/science/article/pii/S0378873304000693>

Newman, M. E. J. (2006). "Modularity and Community Structure in Networks." <https://arxiv.org/abs/physics/0602124>

Grover, A., & Leskovec, J. (2016). "node2vec: Scalable Feature Learning for Networks." <https://arxiv.org/abs/1607.00653>.